P2. COMANDAS PARA MANIPULAR FICHEROS

Sistemas Operativos 1
Marzo 2026

1 INTRODUCCIÓN

En la práctica anterior hemos visto que *bash scripting* es una herramienta fundamental para automatizar tareas y procesar información en sistemas basados en Unix. A través de *scripts*, es posible encadenar operaciones, filtrar datos, transformar formatos y generar informes de manera reproducible y eficiente. Estas competencias forman parte del núcleo de habilidades técnicas que debe dominar un ingeniero informático.

En esta segunda práctica nos centraremos en el tratamiento y análisis de datos relacionados con el ODS 7 "Garantizar el acceso a una energía asequible, segura, sostenible y moderna". Este objetivo promueve el aumento del uso de energías renovables, la mejora de la eficiencia energética y el acceso universal a servicios energéticos fiables. El análisis de indicadores energéticos permite evaluar el progreso hacia estos objetivos y fundamentar decisiones basadas en datos.

Los ejercicios propuestos trabajarán con ficheros en distintos formatos (CSV y texto plano), permitiendo aplicar técnicas de inspección, filtrado, agregación, ordenación y limpieza de datos. Para ello se utilizarán comandos de manipulación de ficheros y procesamiento de texto en entorno Unix, desarrollando tanto la comprensión del funcionamiento de las herramientas como la capacidad de diseñar soluciones estructuradas mediante **tuberías** y **redirecciones**.

El objetivo no es únicamente implementar los *scripts* solicitados, sino comprender las decisiones de diseño adoptadas, el flujo de transformación de la información y las implicaciones de cada paso del procesamiento sobre el resultado final.

2 SCRIPTS A PROGRAMAR

A continuación, se presentan 6 ejercicios, todos con la misma puntación. Todos los scripts necesitan argumentos por líneas de comando, a menos que se indique lo contrario. En caso de que no se introduzcan el número necesario de argumentos, el script deberá mostrar un mensaje de error por pantalla (con el uso esperado) y salir con el código de error `exit 1`. En caso de que se especifiquen todos los argumentos, se puede suponer que se han especificado de forma correcta. Si el script se ejecuta correctamente, será necesario que salga con un código de ejecución correcta, `exit 0`.

Todos los nombres de ficheros, directorios y cadenas a tratar no tendrán espacios ni caracteres especiales. Al campus virtual hay disponible una carpera comprimida (`datos`) que contiene ficheros CSV, TXT y MD que se utilizarán para realizar las pruebas correspondientes.

Para realizar estos ejercicios **está permitido utilizar las instrucciones que hay en el tutorial 1 y tutorial 2** (a menos que se indique lo contrario), así como utilizar opciones de estas instrucciones, aunque no estén incluidas en los tutoriales. Se recomienda investigar las opciones mediante el comando `man`. El uso de comandos no permitidos tendrá una penalización.

Ejercicio 1

Escribe un *script* llamado **ejercicio1.sh** que tome como argumento un fichero a inspeccionar. El *script* debe mostrar el tipo de fichero, directorio dónde se encuentra, nombre, número de líneas, así como las primeras y últimas 3 líneas del fichero. Si es CSV, mostrará también el número de columnas. **P1**. ¿Qué limitación tiene decidir si "es CSV" solo mirando si la primera línea contiene comas?

Formato de uso: `./ejercicio1.sh <fichero>`

Ejemplos de uso:

```
./ejercicio1.sh
Uso: ./ejercicio1.sh <fichero>

./ejercicio1.sh archivo_inexistente.txt
Error: el fichero no existe o no es un fichero regular.


./ejercicio1.sh share-of-final-energy-consumption-from-renewable-sources.csv
Tipo: share-of-final-energy-consumption-from-renewable-sources.csv: CSV text
Directorio: .
Nombre: share-of-final-energy-consumption-from-renewable-sources.csv
Lineas: 5599
Columnas: 4
Primeras 3 lineas:
Entity,Code,Year,Renewable energy share in the total final energy consumption
(%)
Afghanistan,AFG,2000,44.99
Afghanistan,AFG,2001,45.6
Ultimas 3 lineas:
Zimbabwe,ZWE,2020,68.3
Zimbabwe,ZWE,2021,66.82

./ejercicio1.sh the_energy_progress_report_2024.txt
Tipo: the_energy_progress_report_2024.txt: Unicode text, UTF-8 text, with CRLF
line terminators
Directorio: .
Nombre: the_energy_progress_report_2024.txt
Lineas: 13775
Primeras 3 lineas:
TRACKING SDG 7
THE ENERGY
PROGRESS REPORT
Ultimas 3 lineas:
Detailed datasets with country data for all SDG 7 indicators can be accessed at
no charge at
https://trackingsdg7.esmap.org/downloads. The chapters of this report may be
downloaded
individually from the same site
```

Ejercicio 2

Crea un *script* llamado **ejercicio2.sh** que filtre un fichero CSV por año y por un umbral mínimo del valor numérico (última columna). Se debe crear un fichero nuevo (`filtrado_<year>.csv`) con la cabecera y las filas que cumplen las condiciones `Year == <year>` y `valor >= <umbral>`. Además, se debe mostrar por pantalla el número de filas de dicho fichero. **P2**. ¿Qué error introduces si el CSV no tiene exactamente 4 columnas? ¿Cómo lo detectarías con comandos permitidos?

Formato de uso: `./ejercicio2.sh <csv> <year> <umbral>`

Ejemplos de uso:

```
./ejercicio2.sh
Uso: ./ejercicio2.sh <csv> <year> <umbral>

./ejercicio2.sh    share-of-final-energy-consumption-from-renewable-sources.csv
2020 40
Filas (sin cabecera) en filtrado_2020.csv: 64

./ejercicio2.sh    share-of-final-energy-consumption-from-renewable-sources.csv
2015 60
Filas (sin cabecera) en filtrado_2015.csv: 41
```

Ejercicio 3

Desarrolla un *script* llamado **ejercicio3.sh** que muestre el Top N de entidades con mayor porcentaje (última columna) para un año dado. El *script* debe filtrar por `Year`, ordenar descendientemente por valor numérico y mostrar el top N con formato: `valor | Entity | Code | Year`. **P3**. Si dos entidades tienen exactamente el mismo valor, ¿cómo debería resolverse el empate? ¿Tu implementación lo contempla?

Formato de uso: `./ejercicio3.sh <csv> <year> <N>`

Ejemplos de uso:

```
./ejercicio3.sh share-of-final-energy-consumption-from-renewable-sources.csv
Uso: ./ejercicio3.sh <csv> <year> <N>

./ejercicio3.sh    share-of-final-energy-consumption-from-renewable-sources.csv
2021 10
95.540000 | Democratic Republic of Congo | COD | 2021
93.190000 | Liberia | LBR | 2021
91.300000 | Gabon | GAB | 2021
90.950000 | Uganda | UGA | 2021
90.410000 | Somalia | SOM | 2021
89.150000 | Central African Republic | CAF | 2021
87.810000 | Ethiopia | ETH | 2021
87.310000 | Malawi | MWI | 2021
83.890000 | Madagascar | MDG | 2021
82.430000 | Iceland | ISL | 2021
```

Ejercicio 4

Implementa un script llamado **ejercicio4.sh** que limpie y normalice un fichero dado y guarde el resultado en un fichero de salida, también indicado por argumento. Las transformaciones a realizar son: (1) eliminar líneas vacías; (2) eliminar `\r` (final de línea Windows); y quitar espacios alrededor de comas (, → ,). **P4**. ¿Por qué normalizar espacios alrededor de comas ayuda a scripts posteriores?

Formato de uso: `./ejercicio4.sh <fichero_entrada> <fichero_salida>`

Ejemplo de uso:

```
./ejercicio4.sh README.md
Uso: ./ejercicio4.sh <fichero_entrada> <fichero_salida>
```

```
./ejercicio4.sh README.md README_limpio
Fichero entrada OK. Lineas entrada: 6
Fichero salida OK. Lineas salida:  3

./ejercicio4.sh the_energy_progress_report_2024.txt report_limpio.txt
Fichero entrada OK. Lineas entrada: 13775
Fichero salida OK. Lineas salida:  9634
```

Ejercicio 5

Crea un *script* llamado **ejercicio5.sh** que, que, dado un rango de años, calcule la media del valor por Code y muestre un *ranking*. El *script* debe filtrar `year_ini <= Year <= year_fin`; calcular media por `Code` (y mostrar también `Entity`) y ordenar desc por media y mostrar top N siguiento el formato: `media | Code | Entity | n_muestras`. **P5**. ¿Qué pasa si el mismo `Code` aparece con distinto `Entity` en el CSV? ¿Qué valor se conserva y por qué?

Formato de uso: `./ejercicio5.sh <csv> <year_ini> <year_fin> <N>`

Ejemplos de uso:

```
./ejercicio5.sh    share-of-final-energy-consumption-from-renewable-sources.csv
2005 2010
Uso: ./ejercicio5.sh <csv> <year_ini> <year_fin> <N>

./ejercicio5.sh    share-of-final-energy-consumption-from-renewable-sources.csv
2005 2010 5
97.120000 | COD | Democratic Republic of Congo | 6
94.898333 | BDI | Burundi | 6
94.140000 | UGA | Uganda | 6
93.641667 | CAF | Central African Republic | 6
93.461667 | ETH | Ethiopia | 6
```

Ejercicio 6

Realizar un *script* llamado **ejercicio6.sh** que busque (*case-insensitive*) un término en todos los ficheros de un directorio (recursivo) y muestre un ranking por número de ocurrencias. **P6**. Si el término buscado aparece varias veces en la misma línea, ¿cómo debería comportarse tu script? Justifica tu decisión.

Formato de uso: `./ejercicio6.sh <directorio> <termino>`

Ejemplos de uso:

```
./ejercicio6.sh datos
Uso: ./ejercicio6.sh <directorio> <termino>

./ejercicio6.sh directorio_no_existe energy
Error: directorio no existe.

./ejercicio6.sh datos SO1
No se encontraron ocurrencias.

./ejercicio6.sh datos energy
1640 | the_energy_progress_report_2024.txt
2 | share-of-final-energy-consumption-from-renewable-sources.csv
```

```
./ejercicio6.sh datos 2000
242 | share-of-final-energy-consumption-from-renewable-sources.csv
84 | the_energy_progress_report_2024.txt
1 | README.md
```

3 PREGUNTAS GENERALES

P7. ¿Qué comprobaciones mínimas debería hacer un *script* antes de procesar un fichero?

P8. ¿Por qué es importante validar los argumentos recibidos antes de ejecutar cualquier operación?

P9. En un pipeline, ¿Por qué es importante decidir qué operación va primero y cuál después?¿El orden puede cambiar el resultado final?

P10. Si el mismo *script* se ejecuta varias veces, ¿cómo evitar sobrescribir información importante accidentalmente?

4 INFORME

Además de entregar los *scripts* correspondientes a los ejercicios anteriores, se requiere la entrega de un informe sobre el trabajo realizado y con respuestas a las preguntas planteadas. **El informe se debe entregar en formato PDF** (NO se admiten informes en formato doc, docx, odt, etc). Este informe debe incluir una **introducción**, una sección donde se muestran y comentan las **pruebas realizadas y respuestas a las preguntas**, así como una sección con las **conclusiones y valoraciones personales de la práctica.** Importante: **se ha de indicar en una sección la contribución de cada uno de los miembros del grupo, indicando el porcentaje de la nota que merecería cada uno (p.ej., 25%-25%-50%; 50%-50%; 30%-70%).**

El documento ha de tener un **máximo de 6 páginas** (sin contar la portada o índice). Se evaluará las pruebas realizadas (60%), escritura y expresión (10%), así como la estructura y presentación del informe (30%).

5 ENTREGA Y EVALUACIÓN

Cada grupo debe subir un **único archivo ZIP** a la tarea asociada a esta práctica en el campus virtual con el nombre **P2_GrupoXX_nombre1_apellido1_nombre2_apellido2.zip**, y debe incluir los **scripts** y el **informe** solicitado con todas las secciones especificadas en el apartado anterior.

La nota final de la práctica tendrá los siguientes pesos: 60% scripts y 40% informe.